

Resource Management and Circuit Scheduling for Distributed Quantum Computing Interconnect Networks

Sima Bahrani, Romerson D. Oliveira, Juan Marcelo Parra-Ullauri, Rui Wang, and Dimitra Simeonidou

Abstract—Distributed quantum computing (DQC) has emerged as a promising approach to overcome the scalability limitations of monolithic quantum processors in terms of computational capability. However, realising the full potential of DQC requires effective resource management and circuit scheduling. This involves efficiently assigning each circuit to a subset of quantum processing units (QPUs), based on factors such as their computational power and connectivity. In heterogeneous DQC networks with arbitrary connectivity topologies and non-identical QPUs, this becomes a complex challenge. This paper addresses resource management and circuit scheduling in such settings, with a focus on computing resource allocation in a quantum data center. We propose circuit scheduling algorithms based on Mixed-Integer Linear Programming (MILP). Our MILP model accounts for errors arising from inter-QPU communication. In particular, the proposed schemes consider key factors, including network topology, QPU capacities, and quantum circuit structure, to make efficient scheduling and allocation decisions. Simulation results demonstrate that our proposed algorithms significantly improve circuit execution time and scheduling efficiency (measured by makespan and throughput), while also reducing inter-QPU communication overhead, compared to baseline strategies. This work provides valuable insights into resource management strategies for scalable and heterogeneous DQC systems.

Index Terms—Distributed quantum computing, Quantum networks, Resource allocation, Quantum Circuits Scheduling

I. INTRODUCTION

QUANTUM computing has gained attention as a solution for tackling intractable problems, due to its capacity to solve them significantly faster than traditional computers. In recent years, there have been notable advancements

in quantum hardware and control systems, leading to the development of noisy intermediate-scale quantum processing units (QPUs). However, despite these efforts, current quantum processors still remain limited in their computational power [1], [2]. Distributed quantum computing (DQC) aims to harness the collective power of multiple interconnected quantum processors, enabling the execution of larger and more complex quantum algorithms. In DQC, quantum algorithms are partitioned and executed across a network of quantum processors, which are interconnected through both quantum and classical communication channels. By distributing the computational workload among multiple quantum processors, this approach facilitates the development of scalable quantum computing systems that can surpass the limitations imposed by individual quantum processors [3], [4].

DQC is expected to progress through stages of increasing scale and heterogeneity [2]. This progression spans from the integration of multiple quantum processors within a single large quantum computer to the establishment of interconnected quantum processors across various quantum data centres. A quantum data centre, in this context, refers to a facility housing quantum computers at the scale of 10s - 100s meters, like a classical data centre. One of the fundamental requirements for DQC is the ability to perform quantum operations between distant qubits located on separate QPUs, often referred to as *remote gates*. Such operations typically rely on three components: remote entanglement generation, local quantum operations at each QPU, and classical communication between them [3], [5]. As an illustrative example, Fig. 1 shows the circuit for implementing a controlled-NOT (CNOT) gate between two computing qubits, q_{cp1} and q_{cp2} , residing on QPU₁ and QPU₂, respectively. The procedure begins with generating entanglement between dedicated communication qubits, q_{cm1} and q_{cm2} , forming a shared entangled qubit pair, commonly referred to as an entangled bit (*ebit*). The Pauli-X gate operation on QPU₂ and the Z gate operation on QPU₁ are then conditioned on classical measurement outcomes exchanged between the QPUs. This approach generalises to other controlled-unitary operations.

The integration of quantum networking and computation in DQC requires coordinated resource management across both domains. Compute resources refer to the computing qubits within QPUs, while communication resources include communication qubits and networking components such as optical switches. In this work, we assume that the network reliably establishes entanglement links to support remote operations

This work was supported by Engineering and Physical Sciences Research Council Quantum Communication Hub grant ref. EP/T001011/1, EPSRC Integrated Quantum Networks Hub grant EP/Z533208/1, International Science Partnerships Fund from Science and Technology Facilities Council (STFC) DyMEND project (grant ref:593) and the Innovate UK-funded project, Quantum Data Centre of the Future (10004793). (Corresponding authors: Sima Bahrani and Rui Wang)

Sima Bahrani, Rui Wang, and Dimitra Simeonidou are with the Smart Internet Lab, School of Electrical, Electronic, and Mechanical Engineering, University of Bristol, Bristol, BS8 1UB, UK (e-mail: sima.bahrani@bristol.ac.uk; rui.wang@bristol.ac.uk; Dimitra.Simeonidou@bristol.ac.uk).

Romerson D. Oliveira was with the Smart Internet Lab, School of Electrical, Electronic, and Mechanical Engineering, University of Bristol, Bristol, BS8 1UB, UK, and is now with Nu Quantum, Cambridge, UK (e-mail: romerson.oliveira@nu-quantum.com).

Juan Marcelo Parra-Ullauri was with the Smart Internet Lab, School of Electrical, Electronic, and Mechanical Engineering, University of Bristol, Bristol, BS8 1UB, UK, and is now with BT, Bristol, UK (e-mail: juan.m.parraullauri@bt.com).

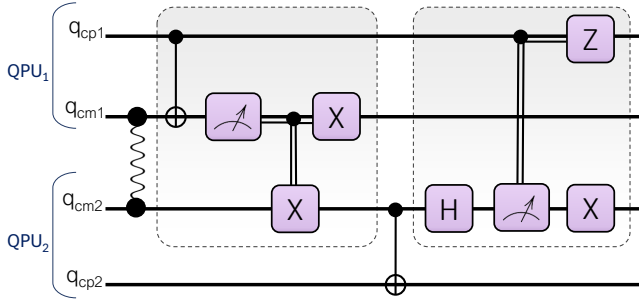


Fig. 1. Circuit diagram of remote CNOT gate performed between two QPUs.

and allocates communication resources on demand. Focusing on compute resource management, we conceptually distinguish two key components. The first jointly determines QPU allocation and the partition count per circuit under multiple concurrent circuits; we refer to this decision process as *circuit scheduling*. The second component, *circuit partitioning*, performs QPU–circuit mapping at a finer granularity by assigning the circuit qubits to the allocated QPUs according to the chosen partition count, while accounting for QPU capacities and the circuit’s interaction structure. In this work, we focus primarily on the scheduling layer, while explicitly capturing its interdependence with circuit partitioning by incorporating partition granularity (i.e., the number of partitions per circuit) as a decision variable in the optimisation framework.

Job scheduling in classical cloud data centres is a well-studied problem, typically focusing on objectives such as maximising resource utilisation, minimising makespan (i.e., the total completion time of all tasks), and balancing workload across computing resources [6]–[9]. In such environments, scheduling decisions primarily influence job execution time and system-level efficiency metrics. However, DQC introduces additional critical considerations. In particular, remote gate operations introduce infidelities that can degrade computational accuracy. These infidelities arise from qubit decoherence during remote gate execution as well as from imperfect entanglement generation between QPUs. Since scheduling decisions determine how circuits are distributed across QPUs, they directly influence the frequency and quality of inter-QPU entanglement link usage. Consequently, in DQC, scheduling decisions affect not only circuit execution time but also the computational fidelity of quantum tasks, inter-QPU communication overhead, and overall system-level performance metrics such as makespan. In addition, DQC scheduling must satisfy *capacity constraints*, requiring that the selected QPUs provide sufficient combined physical qubits to meet a circuit’s qubit demand. This further increases the complexity of the decision process. Furthermore, prior studies [10] have shown that quantum circuits exhibit varying degrees of sensitivity to errors introduced by distributed execution. For example, if a sparsely connected circuit is scheduled first and occupies the most favourable QPUs, such as those with high-fidelity links or larger qubit capacities, subsequent densely connected circuits may be forced into fragmented or suboptimal allocations. This can increase overall inter-QPU communication overhead and degrade execution performance. These characteristics motivate the design of circuit scheduling strategies that jointly consider

qubit-capacity feasibility, circuit structure, and inter-QPU link fidelity under multi-circuit contention.

Beyond the considerations mentioned above, unlike classical scheduling where job execution time can typically be estimated prior to resource allocation, in DQC, the execution time of a quantum circuit depends on how it is partitioned across QPUs, which is itself determined by the scheduling decision. This tight coupling makes proper prior estimation challenging and motivates the adoption of lightweight surrogate models, the regression-based partitioning cost estimation in Section III.

While extensive research has addressed various aspects of DQC, including network resource management, quantum compilation, and circuit partitioning, comparatively fewer works have focused on QPU allocation and circuit scheduling. Several studies have addressed network resource management and scheduling, focusing on network-level operations such as entanglement request scheduling, quantum routing, and efficient entanglement generation or allocation protocols [11]–[14]. In contrast, our work addresses compute resource management, specifically Quantum Circuit (QCirc) scheduling and QPU–QCirc mapping in heterogeneous DQC networks.

Quantum compilation and circuit partitioning for DQC have been studied extensively. Prior works have addressed compiler design challenges and optimisation of circuit depth and remote entanglement overhead under network and hardware constraints [15]–[17]. In parallel, graph- and hypergraph-based techniques have been proposed to partition QCircs so as to minimise remote entanglement overhead across distributed QPUs [18]–[23]. Beyond partitioning, Andres et al. [24] considered circuit placement and distribution over heterogeneous networks with arbitrary topologies, integrating mapping decisions within a compiler framework. However, these approaches focus on optimising DQC for a single QCirc and do not explicitly address multi-circuit resource contention in heterogeneous DQC systems. Parekh et al. [25] proposed a greedy scheduling algorithm that assigns QCircs sequentially to available QPUs. However, their approach does not jointly account for network topology, QPU decoherence properties, and circuit-specific structural characteristics within a unified optimisation framework.

Here, we address the problem of *circuit scheduling* in DQC networks, accounting for quantum-specific challenges including fidelity degradation due to remote operations. We consider a general heterogeneous network model with QPUs of varying capacities, arbitrary connectivity topologies, and diverse QCircs. Within this framework, we propose two scheduling algorithms: (i) a dynamic batch scheduling approach that leverages mixed-integer linear programming (MILP) to jointly optimise QPU allocation and partition count for a batch of QCircs, and (ii) an MILP-based online scheduling strategy that allocates circuits individually. The proposed methods are analysed and evaluated through detailed simulations. Our results indicate that quantum-aware, batch-level optimisation can reduce inter-QPU communication overhead and improve system-level efficiency, particularly in heterogeneous network settings. The analysis also highlights a trade-off between makespan and communication cost, offering useful design insights for emerging quantum data centre architectures.

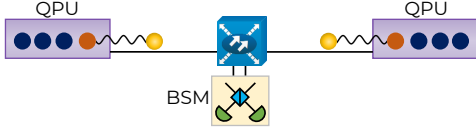


Fig. 2. High-level schematic of remote entanglement generation between a pair of QPUs via an intermediate BSM module. Blue circles denote computing qubits, and the brown circle denotes the communication qubit.

The remainder of this paper is organised as follows. Sec. II describes the DQC network model. Section III presents the proposed Batch-QCirc and Single-QCirc scheduling methods. Section IV evaluates the proposed methods through simulations. Finally, Section V concludes the paper.

II. DQC NETWORK MODEL

We investigate a DQC network within a quantum data centre, where QPUs are interconnected via both quantum and classical links. Promising architectures for such data centre networks will employ low-loss high-speed optical switches to dynamically establish quantum links between QPUs. Quantum information of stationary computing qubits can be encoded in photons via transducers, using degrees of freedom such as time-bin, polarization, frequency, etc [26], [27]. At the physical and link layers, remote entanglement between two QPUs can be generated in various ways [28], with one example illustrated in Fig. 2. This method relies on a Bell state measurement (BSM) module placed between the QPUs, enabling heralded entanglement generation. From an architectural perspective, switch-based data centre topologies such as fat-trees are promising candidates. A representative example with four pods is shown in Fig. 3(a), where QPUs are interconnected through three layers of optical switches to provide all-to-all connectivity. Alternatively, some designs may leverage entanglement swapping, allowing a QPU to act as a repeater and connect QPUs that are not directly linked.

At a higher abstracted connectivity level, the DQC network can be represented as in Fig. 3(b), which highlights the key components: QPUs connected by quantum and classical links, coordinated by a Quantum Network Controller. This abstraction captures the essential elements of the network while remaining agnostic to specific physical implementations and architectural details.

On top of the physical and logical connectivity layers, the DQC architecture can be understood through its compilation and execution workflow, as illustrated in Fig. 4. A core element of quantum computing is quantum compilation, which translates a high-level quantum program into hardware-level instructions [29], [30]. This translation is performed through a layered compilation stack (left side of Fig. 4). In the distributed setting, additional tasks arise at the level of the intermediate quantum representation, which we refer to as QCirc (right side of Fig. 4). These tasks involve resource management across both compute and network domains. Compute resources correspond to the physical qubits embedded in QPUs, whereas communication resources include communication qubits and networking components such as optical switches.

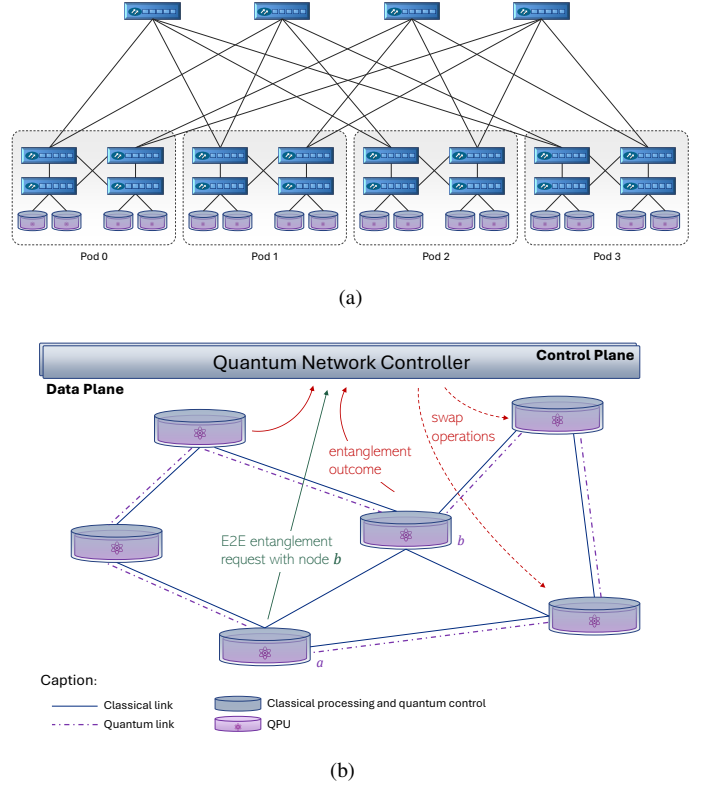


Fig. 3. (a) Example network architecture for a DQC network within a quantum data centre (fat-tree topology with 4 pods). (b) High-level abstract representation of the DQC network model.

We now turn to the key definitions and assumptions related to our DQC network model. First of all, we abstract the network into a logical graph model $G(V, E)$, where QPUs are represented as nodes and quantum logical links as edges. An edge between QPU_{j_1} and QPU_{j_2} is characterised by the entanglement establishment latency $T_{j_1 j_2}$ (i.e., the average time required for one successful entanglement generation event between QPUs) and the fidelity $F_{j_1 j_2}$ of the generated entangled pair. This model relies on several simplifying assumptions. First, we assume on-demand entanglement generation: remote entanglement is initiated and completed at the start of a remote operation, and the resulting entangled pair is used immediately, eliminating the need for long-term entangled-pair storage. Second, the latency and fidelity parameters are assumed to be fixed for each logical link. Under these modelling assumptions, $T_{j_1 j_2}$ captures the primary communication-induced timing overhead of distributed execution. It is primarily determined by the entanglement generation rate, which typically decreases with increasing link loss. For example, in the fat-tree topology depicted in Fig. 3(a), link loss increases with the number of optical switches in the path, thereby increasing remote gate execution time. Link's fidelity $F_{j_1 j_2}$ is similarly influenced by factors such as link loss and the specific entanglement generation protocol used.

The set of QPUs in the network is denoted by $\mathcal{QP} = \{QPU_j\}_{j=1}^J$, where J represents the total number of QPUs in the network. In the graph model $G(V, E)$, the vertex set V corresponds to $\mathcal{QP}$, i.e., $V \equiv \mathcal{QP}$. The capacity of QPU_j , defined as its total number of computing qubits, is denoted by

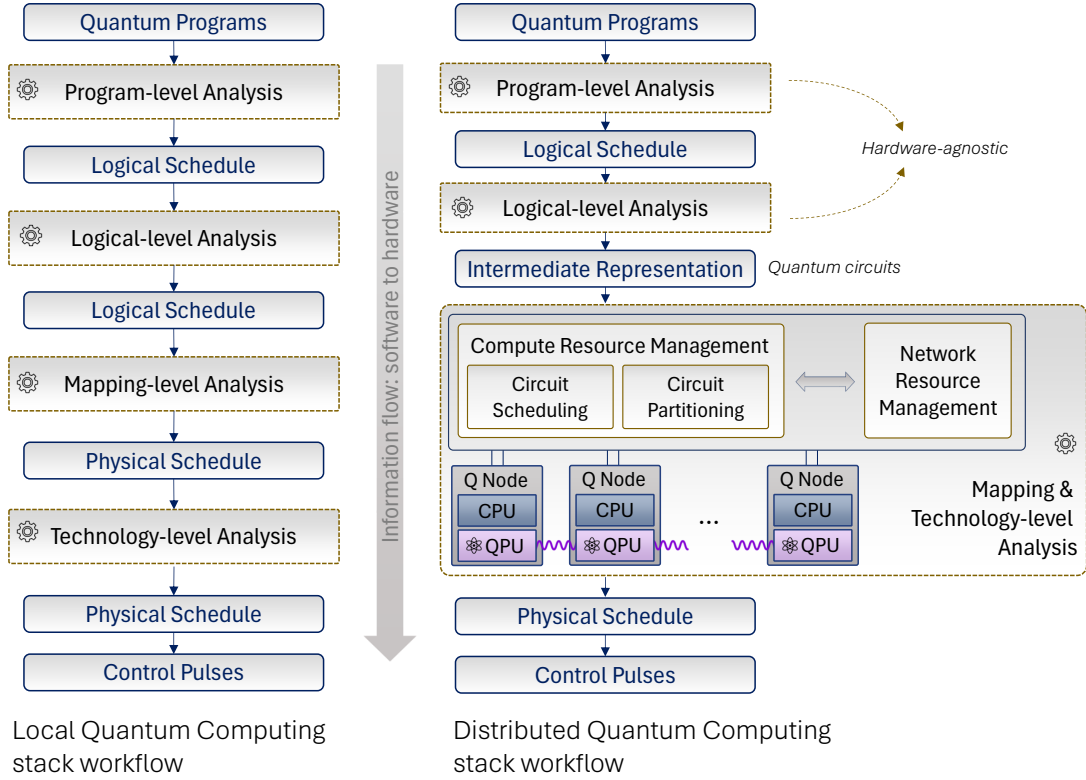


Fig. 4. Workflow of the quantum stack for both local and distributed quantum computing. A layered-oriented approach for compilation tools that bridge quantum algorithms with quantum devices. The stack workflow for local quantum computing, depicted on the left side of the figure, is based on [30].

N_j . We also introduce a binary parameter s_j , which represents the current availability of QPU_j . Specifically, $s_j = 1$ if QPU_j is available, and $s_j = 0$ otherwise. We assume that all QPUs are based on the same technology and exhibit nearly identical decoherence properties, characterised by a common decoherence time parameter, T_{dec} . It is important to note that the concept of “decoherence time” is technology-dependent. For example, in superconducting-qubit systems, two parameters, energy relaxation time T_1 and dephasing time T_2 , are often reported; these can be combined into a single effective value, such as $\min(T_1, T_2)$ or $\frac{T_1 T_2}{T_1 + T_2}$, for modelling purposes. In trapped-ion systems, a memory lifetime parameter may be used instead. Regardless of the specific definition, there always exists a parameter (or suitable combination thereof) that captures the temporal limitations imposed by decoherence. To maintain platform independence, we abstract this into a single representative parameter T_{dec} in our model.

III. PROPOSED QCIRC SCHEDULING METHODS

In this section, we consider the problem of QCirc scheduling for the DQC network described above. Our primary objective is to schedule a set of quantum circuits and allocate a subset of QPUs to each of them. The set of input quantum circuits is denoted by $\mathcal{QC} = \{\text{QCirc}_m\}_{m=1}^M$, where M is the total number of circuits. The required number of qubits for each circuit $\mathcal{QC} = \{\text{QCirc}_m\}$ is represented by w_m . We define K_m^{max} as the maximum allowable number of partitions for QCirc_m , determined according to factors such as data centre policy or circuit size. This parameter bounds circuit fragmentation,

limits excessive inter-QPU communication, and captures practical fidelity constraints and system-level policy requirements. Throughout this work, we assume a non-preemption model, meaning that once a QCirc begins execution on its allocated QPUs, it runs to completion without interruption. Furthermore, each QPU can host at most one partition at any given time, meaning it cannot concurrently execute partitions from different circuits. In the following, we propose two primary QCirc scheduling methods. The first method, termed *Batch-QCirc Scheduling*, is based on dynamic batch scheduling, where a group of circuits is selected and assigned to QPUs collectively. The second method, referred to as *Single-QCirc Scheduling*, assigns each circuit individually.

A. Dynamic Batch QCirc Scheduling

To efficiently schedule diverse quantum circuits (e.g. quantum Fourier transform), with varying qubit requirements and structures, we propose a batch scheduling framework that jointly optimises QPU allocation and the number of partitions for a batch of circuits. The algorithm operates in repeated scheduling cycles, each consisting of four main stages: (a) batch selection, where incoming quantum circuits are grouped into a batch based on their combined qubit requirements; (b) batch assignment, where the selected batch is mapped to available QPUs via a holistic MILP-based optimisation that accounts for decoherence effects and inter-QPU connectivity; (c) overflow handling, where any circuits that could not be assigned in the main step are promptly scheduled to run as soon as resources become available; and (d) filling idle

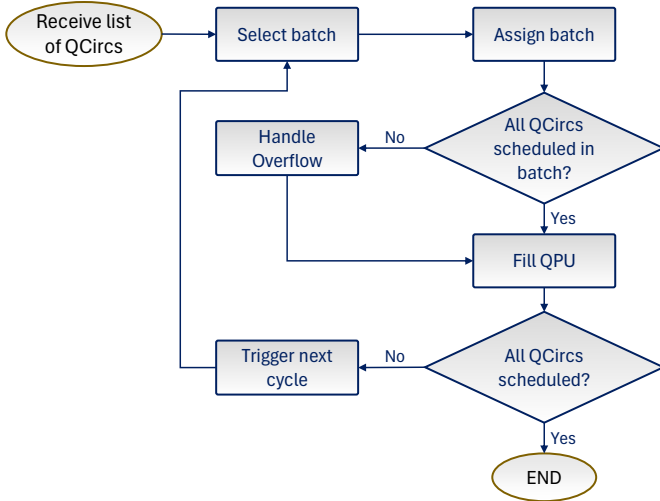


Fig. 5. Workflow of the Dynamic Batch-QCirc Scheduling method.

QPUs, where any QPUs that remain unallocated after the batch assignment process are identified and, if suitable candidates exist in the remaining circuits of QC , assigned to them, enhancing the utilisation of available resources.

The workflow for the proposed batch scheduling method is illustrated in Fig. 5, and the corresponding algorithm pseudocode is outlined in Algorithm 1. In each scheduling cycle, a batch of circuits is selected from QC . This batch is denoted by $B = \{QCirc_m \mid m \in \mathcal{I}\}$, where $\mathcal{I} \subseteq \{1, \dots, M\}$ is the set of indices corresponding to the selected circuits. The batch size, $S = |\mathcal{I}|$, may vary across scheduling cycles.

We adopt a simple batch selection process, where elements of QC are sequentially added to B based on their requests' arrival time until the total required number of qubits, $c_{req} = \sum_{m \in \mathcal{I}} w_m$, exceeds a threshold βc_{tot} . Here, c_{tot} denotes the total qubit capacity of all available QPUs, given by $c_{tot} = \sum_j s_j N_j$. The parameter β , fixed in the range $0 < \beta < 1$, is chosen to ensure that, with high probability, all circuits in the selected batch can be successfully assigned to available QPUs.

The selected batch then undergoes a batch assignment process, referred to as `ASSIGNBATCH` in Algorithm 1, where the circuits are mapped to the available QPUs. We have developed an MILP-based batch assignment algorithm, which will be described in detail in Sec. III-A1.

After the batch assignment, there might be a few circuits unassigned if the β is large, e.g., close to 1, or some circuits are complex to be distributed among available QPUs given the constraints. Any circuits in the batch that remain unassigned are handled after batch assignment using the `HANDLEOVERFLOW` procedure, described in Algorithm 2. The QPUs are first sorted in ascending order based on their expected busy time (EBT). The unassigned circuits are then assigned sequentially to the sorted QPUs, while taking into account circuit constraints such as capacity and the maximum number of partitions. The EBT for a given QPU can be estimated from the execution time of circuits already assigned to it or, when such information is not available, from other factors such as circuit connectivity, the number of partitions assigned to the circuit, and historical assignment data. This overflow step acts

Algorithm 1 Dynamic Batch-QCirc Scheduler

```

1: Input:  $QP, QC, G$ 
2: while not ISEMPTY( $QC$ ) do
3:    $batch \leftarrow \text{SELECTBATCH}(QC, \beta)$ 
4:    $batchMap \leftarrow \text{ASSIGNBATCH}(batch, QP, G)$ 
5:   if not ALLASSIGNED( $batch, batchMap$ ) then
6:     HANDLEOVERFLOW( $batch, batchmap, QP$ )
7:   end if
8:   REMOVEASSIGNED( $QC, batch$ )
9:   if ANYAVAILABLE( $QP$ ) then
10:    FILLQPU( $QC, QP$ )
11:   end if
12:   TRIGGERNEXTCYCLE( $QP, \alpha$ )
13: end while
  
```

primarily as a failsafe, as β is chosen to make such cases rare.

To improve QPU utilisation, we incorporate a filling mechanism that assigns circuits to idle, unallocated QPUs. The `FILLQPU` algorithm, described in Algorithm 3, selects circuits from QC and assigns each of them to idle QPUs individually, using a MILP-based optimisation strategy. To reduce the risk of excessive remote gate generation, only circuits with a ν_m value below a specified threshold are considered. Here, ν_m (also see in Sec. III-A1) is a score parameter that quantifies the circuit's connectivity and its likelihood of generating costly remote gates in a distributed setting. The MILP-based Single-QCirc assignment process (`ASSIGNQCIRC` in Algorithm 3) is detailed in Sec. III-B.

Once all QPUs are allocated, the algorithm waits until a fraction α of the total qubit capacity across all QPUs, i.e., $\alpha \sum_j N_j$, becomes available, then proceeds to the next scheduling cycle. This mechanism is referred to as `TRIGGERNEXTCYCLE` in Algorithm 1.

The parameters β and α control two distinct phases of the scheduling cycle with complementary roles in shaping system-level performance. The parameter β controls batch formation prior to optimisation by limiting the aggregate qubit demand relative to the available system capacity, thereby reducing the likelihood of infeasibility, while α determines the temporal progression of successive scheduling cycles. Both parameters affect the scope of joint optimisation within a cycle. Setting these parameters appropriately is therefore important: too large a β risks infeasibility, while too large an α introduces unnecessary idle time between cycles, as confirmed by the simulation results in Sec. IV, where $\alpha = 0.75$ yields worse makespan than $\alpha = 0.55$.

1) Batch-QCirc Assignment: We propose an optimisation algorithm to minimise the detrimental effects of inter-QPU communication, focusing on decoherence-induced errors and fidelity loss from remote entanglement generation. Remote gates incur significantly higher execution time than local gates due to quantum and classical communication delays and loss, increasing qubit exposure to decoherence. Moreover, remote entanglement generation is inherently imperfect, with fidelity that can degrade further through operations such as entanglement swapping and distillation. Repeated use of remote gates amplifies these effects, causing substantial fidelity

Algorithm 2 HANDLEOVERFLOW

```

1: Input: batch, batchMap,  $\mathcal{QP}$ ,  $G$ 
2:  $U \leftarrow$  Unassigned QCircs in batch
3:  $\mathcal{QP}_s \leftarrow$  QPUs in  $\mathcal{QP}$  sorted by increasing EBT
4: for QCircm in  $U$  do
5:   Scan  $\mathcal{QP}_s$  to find a set  $\{\text{QPU}_{j_k}\}_{k=1}^K$ 
     such that  $\sum_{k=1}^K N_{j_k} \geq w_m$  and  $K \leq K_m^{\max}$ 
6:   Assign QCircm to  $\{\text{QPU}_{j_k}\}_{k=1}^K$ 
7:    $\mathcal{QP}_s \leftarrow \mathcal{QP}_s \setminus \{\text{QPU}_{j_k}\}_{k=1}^K$  remove allocated QPUs
8: end for

```

Algorithm 3 FILLQPU

```

1: Input:  $\mathcal{QC}$ ,  $\mathcal{QP}$ ,  $G$ 
2:  $avlCapacity \leftarrow \sum_j N_j$  for idle QPUs
3: if  $avlCapacity > 0$  then
4:   for each QCircm in  $\mathcal{QC}$  do
5:     if  $\nu_m \leq \Gamma$  and  $w_m \leq avlCapacity$  then
6:       ASSIGNQCIRC(QCircm)
7:       REMOVEASSIGNED(QCircm)
8:       Update  $avlCapacity$ 
9:     end if
10:  end for
11: end if

```

loss and degrading the overall computational accuracy of distributed quantum computations. Our approach mitigates this degradation for the batch of QCircs as a group. The output of the Batch-QCirc assignment specifies the number of partitions assigned to each QCirc and the QPUs allocated for their execution, which then guide the circuit partitioning process in efficiently decomposing each QCirc into smaller sub-circuits. To mathematically formulate the problem of Batch-QCirc assignment, we begin by defining the following decision variables:

- Let r_{mj} be a binary variable, where $r_{mj} = 1$ if QPU_{*j*} is allocated to QCirc_{*m*}, and $r_{mj} = 0$ otherwise.
- Let y_{mk} be a binary variable, where $y_{mk} = 1$ if QCirc_{*m*} is partitioned into exactly k parts (i.e., allocated to k distinct QPUs), and $y_{mk} = 0$ otherwise.

We define a cost function that captures the impact of qubit decoherence and imperfect entanglement generation during remote gate executions as follows:

$$C = \sum_{m \in \mathcal{I}} \sum_{k=1}^{K_m^{\max}} \sum_{1 \leq j_1 < j_2 \leq J} \nu_{mk} y_{mk} r_{mj_1} r_{mj_2} \left(\omega_0 \frac{w_m T_{j_1 j_2}}{T_{\text{dec}}} + \omega_1 (1 - F_{j_1 j_2}) \right) \quad (1)$$

where $T_{j_1 j_2}$ is the latency parameter, i.e., average time required to establish entanglement, associated with a remote gate between QPU_{*j*}₁ and QPU_{*j*}₂, and $F_{j_1 j_2}$ is the fidelity parameter associated with remote entanglement generation for the QPU pair. The term $(w_m T_{j_1 j_2})/T_{\text{dec}}$ captures the relative decoherence impact, where larger circuits (w_m) and longer communication delays ($T_{j_1 j_2}$) increase decoherence relative to the characteristic decoherence parameter T_{dec} . The parameters ω_0 and ω_1 are constant weights that reflect the relative

TABLE I
LIST OF KEY NOTATIONS AND PARAMETERS

Parameter	Definition
$G(V, E)$	Graph representation of DQC network
$\mathcal{QP}$	Set of all QPUs
$\mathcal{QC}$	Set of all QCircs
$\mathcal{B}$	Selected batch of QCircs
J	Number of QPUs
M	Number of QCircs
S	Number of QCircs in the selected batch
T_{dec}	Decoherence time parameter of QPUs
N_j	Number of computing qubits within QPU _{<i>j</i>}
s_j	Equals to 1 if QPU _{<i>j</i>} is available, and 0 otherwise
w_m	Number of qubits required for QCirc _{<i>m</i>}
$K_m^{\max}$	Partition count threshold for QCirc _{<i>m</i>}
r_{mj}	Binary variable equal to 1 if QCirc _{<i>m</i>} is assigned to QPU _{<i>j</i>} , and 0 otherwise
y_{mk}	Binary variable equal to 1 if QCirc _{<i>m</i>} is partitioned to k parts, and 0 otherwise
$T_{j_1 j_2}$	The latency parameter for establishing entanglement for remote gate execution, involving QPU _{<i>j</i>} ₁ and QPU _{<i>j</i>} ₂
$F_{j_1 j_2}$	The fidelity parameter associated with remote entanglement generation between QPU _{<i>j</i>} ₁ and QPU _{<i>j</i>} ₂
ν_{mk}	Partitioning cost coefficient for QCirc _{<i>m</i>} and a given partition count k
g_m	Average degree of nodes in the graph model of QCirc _{<i>m</i>}
σ_m	Std. dev. of node degrees in the graph model of QCirc _{<i>m</i>}
$\lambda_{2,m}$	Algebraic connectivity of the graph model of QCirc _{<i>m</i>}
γ_m	Weighted density of the graph model of QCirc _{<i>m</i>}

importance of decoherence-induced errors versus fidelity loss from entanglement generation; unless otherwise stated, we set $\omega_0 = \omega_1$.

To incorporate circuit structure and connectivity patterns into our MILP objective, we introduce a partitioning cost coefficient ν_{mk} , which captures the structural overhead induced by distributing QCirc_{*m*} across k QPUs and thereby enables circuit-structure-aware resource allocation. Each quantum circuit is represented as a weighted undirected graph, where nodes correspond to qubits and edges correspond to two-qubit gates. The weight of each edge is given by the number of occurrences of gates between the respective qubit pair. Based on this representation, ν_{mk} is defined as the estimated graph cut size under k -way partitioning (with partitions of equal or nearly equal size). Although many distinct k -partitions are possible, adopting a standard balanced k -partition is sufficient to capture a circuit's connectivity level and its tendency to generate costly remote gates, while avoiding extra complexity.

To efficiently obtain ν_{mk} for different circuits without performing explicit graph partitioning, we propose an estimation method based on a linear regression model trained on three structural graph features: weighted density, algebraic connectivity, and coefficient of variance. This regression provides a lightweight estimate of cut size across circuit types, thereby making ν_{mk} tractable to compute in our MILP formulation. The three features are defined as follows:

(i) **Weighted density** (γ_m) quantifies the average connectivity per qubit pair, computed as

$$\gamma_m = \frac{\frac{1}{2} \sum_{i=1}^{w_m} g_m^{(i)}}{\binom{w_m}{2}}, \quad (2)$$

where $g_m^{(i)}$ denotes the weighted degree of qubit node i . The numerator corresponds to the total edge weight of the circuit

graph, i.e., the total number of two-qubit gate occurrences across the circuit.

(ii) **Algebraic connectivity** ($\lambda_{2,m}$) is the second-smallest eigenvalue of the normalised Laplacian matrix L_m of the circuit graph [31]. It measures global connectivity strength; larger values indicate that the graph is harder to partition without cutting many edges.

(iii) **Coefficient of variance** (σ_m/g_m) captures connectivity imbalance across qubits, defined as

$$\frac{\sigma_m}{g_m} = \frac{\sqrt{\frac{1}{w_m} \sum_{i=1}^{w_m} (g_m^{(i)} - g_m)^2}}{\frac{1}{w_m} \sum_{i=1}^{w_m} g_m^{(i)}}, \quad (3)$$

where g_m and σ_m are the average and standard deviation of the weighted degrees $\{g_m^{(i)}\}$, respectively.

These features jointly capture complementary aspects of circuit structure: γ_m reflects the overall connectivity strength, particularly the density of pairwise interactions; $\lambda_{2,m}$ indicates how robustly the graph remains connected under partitioning; and σ_m/g_m captures connectivity imbalance across qubits that may create partitioning bottlenecks. Importantly, these features scale smoothly with circuit size w_m , enabling the simple but effective **linear regression model** to learn structural tendencies rather than raw size.

The regression target is the normalised k -way cut size:

$$\tilde{\nu}_{mk} = \frac{\text{Cut}_m^{(k)}}{\frac{1}{2} \sum_{i=1}^{w_m} g_m^{(i)}}, \quad (4)$$

where $\text{Cut}_m^{(k)}$ is the sum of edge weights for inter-partition edges when circuit m is optimally partitioned into k balanced parts. We then rescale by the total edge weight to recover the estimated partitioning cost coefficient:

$$\nu_{mk} \approx \left(\hat{\chi}_{0,k} \gamma_m + \hat{\chi}_{1,k} \lambda_{2,m} + \hat{\chi}_{2,k} \frac{\sigma_m}{g_m} + \hat{\chi}_{3,k} \right) \cdot \left(\frac{1}{2} \sum_{i=1}^{w_m} g_m^{(i)} \right). \quad (5)$$

Here, $(\hat{\chi}_{0,k}, \hat{\chi}_{1,k}, \hat{\chi}_{2,k}, \hat{\chi}_{3,k})$ denote the regression coefficients, which are pre-learned and fitted by minimising the mean squared error (MSE)

$$\frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \left(\tilde{\nu}_{dk} - (\hat{\chi}_{0,k} + \hat{\chi}_{1,k} \gamma_d + \hat{\chi}_{2,k} \lambda_{2,d} + \hat{\chi}_{3,k} \frac{\sigma_d}{g_d}) \right)^2, \quad (6)$$

for a given k partitions. In this expression, $\mathcal{D}$ denotes a set of quantum circuits with varying types and qubit counts, and $\tilde{\nu}_{dk}$ is the normalised k -way cut size for circuit $d \in \mathcal{D}$.

Our optimisation problem, which minimises the cost function C in (1), is formulated as the MILP in Formulation 1. The cost function is linearised by introducing the auxiliary variables $z_{mj_1j_2} = r_{mj_1} r_{mj_2}$ and $x_{mkj_1j_2} = z_{mj_1j_2} y_{mk}$. The role of each constraint in Formulation 1 is as follows:

(1) Minimum assignment: Enforces a lower bound on the number of circuits scheduled in the current cycle. By default, $\zeta = M$ so that all circuits in the batch are assigned. If the MILP becomes infeasible, ζ is decremented until a feasible allocation is found.

(2) Capacity requirement: Ensures that the aggregate qubit capacity of the QPUs assigned to QCirc_m meets or exceeds

its qubit requirement w_m . While s_j (QPU availability binary parameter) may vary between scheduling cycles, it is fixed within each cycle and thus treated as a parameter in the MILP.

(3) Single-partition per QPU: Ensures that each QPU is assigned at most one circuit partition in a scheduling cycle.

(4) Link between binary variables: Provides an MILP-compatible link between r_{mj} and y_{mk} .

(5)–(6) Linearisation of $z_{mj_1j_2}$: Implements MILP-compatible linear formulations of the equalities $z_{mj_1j_2} = r_{mj_1} r_{mj_2}$ and $x_{mkj_1j_2} = z_{mj_1j_2} y_{mk}$.

Formulation 1. MILP-based Batch-QCirc assignment problem

$$\min \left\{ \sum_{m \in \mathcal{I}} \sum_{k=1}^{K_m^{\max}} \sum_{1 \leq j_1 < j_2 \leq J} \nu_{mk} x_{mkj_1j_2} \left(\omega_0 \frac{w_m T_{j_1j_2}}{T_{\text{dec}}} + \omega_1 (1 - F_{j_1j_2}) \right) \right\}$$

s.t.

$$(1) \sum_{m \in \mathcal{I}} \sum_{k=1}^{K_m^{\max}} y_{mk} = \zeta$$

$$(2) \sum_{j=1}^J r_{mj} s_j N_j \geq w_m \sum_{k=1}^{K_m^{\max}} y_{mk} \quad \forall m \in \mathcal{I}$$

$$(3) \sum_{j=1}^J r_{mj} = \sum_{k=1}^{K_m^{\max}} k y_{mk} \quad \forall m \in \mathcal{I}$$

$$(4) \sum_{m \in \mathcal{I}} r_{mj} \leq 1 \quad \forall j$$

$$(5) r_{mj_1} + r_{mj_2} - 1 \leq z_{mj_1j_2} \leq r_{mj_1}, r_{mj_2} \\ \forall m \in \mathcal{I}, j_1 < j_2$$

$$(6) z_{mj_1j_2} + y_{mk} - 1 \leq x_{mkj_1j_2} \leq z_{mj_1j_2}, y_{mk} \\ \forall m \in \mathcal{I}, k, j_1 < j_2$$

In the FILLQPU algorithm (Algorithm 3), we employ the parameter ν_{mk} as a scoring metric for circuit connectivity and remote-gate overhead estimation. In particular, we consider the case $k = 2$ (with the index k omitted in Algorithm 3 for simplicity), and only circuits with ν_m not exceeding a threshold Γ are selected for individual assignment. This prevents highly connected circuits from being handled by FILLQPU, which lacks an optimisation mechanism, and reduces the risk of excessive remote gate overhead at this stage.

Algorithm 4 Single-QCirc Scheduler

```

1: Input:  $\mathcal{QC}, \mathcal{QP}, G$ 
2: while not ISEMPTY( $\mathcal{QC}$ ) do
3:   for each  $\text{QCirc}_m$  in  $\mathcal{QC}$  do
4:     ASSIGNQCIRC( $\text{QCirc}_m$ )
5:     REMOVEASSIGNED( $\text{QCirc}_m$ )
6:   end for
7: end while

```

As can be seen from the Batch-QCirc scheduling workflow, the framework incorporates a multi-layer feasibility handling mechanism to ensure robustness. Although batch selection limits the total qubit demand to a fraction β of the combined capacity of the available QPUs, infeasibility may still arise due to MILP constraints such as partition count limits. To address this, the MILP includes the adaptive parameter ζ , which specifies the number of circuits to be assigned in the current batch. If the problem is infeasible with ζ equal to the batch size, ζ is iteratively reduced until a feasible solution is obtained. Any remaining circuits are subsequently processed by the HANDLEOVERFLOW procedure (Algorithm 2). Empirically, β can be chosen to make such overflow events rare; in the experiments reported in Section IV, $\beta = 0.85$ was found to be a good operating point. Together, these steps guarantee that the scheduler consistently produces a feasible allocation and avoids deadlock.

It is worth noting that while this work assumes a homogeneous decoherence parameter T_{dec} across all QPUs, the framework can be naturally extended to systems where QPUs exhibit heterogeneous decoherence times. In particular, T_{dec} in the objective function in (1) can be replaced with an effective coherence time, where for QCirc_m partitioned into k parts:

$$\frac{1}{T_{mk}^{\text{eff}}} = \frac{1}{k} \sum_j \frac{r_{mj}}{T_j^{\text{dec}}}, \quad (7)$$

which is the harmonic mean of the decoherence times of the k allocated QPUs. This captures the fact that a remote gate between a QPU pair introduces decoherence to all of the circuit's qubits on the allocated QPUs. Such a replacement preserves the linear structure of the MILP, since $1/k$ is a precomputed constant for each k , and the multiplicative binary variable r_{mj} can be linearised following the same approach described above. A detailed quantitative evaluation of heterogeneous decoherence settings constitutes an interesting direction for future investigation.

B. Single-QCirc Scheduling

Another approach for scheduling quantum circuits is to schedule them individually rather than in batches. In this case, we propose a single-QCirc scheduling algorithm, presented in pseudocode in Algorithm 4. This algorithm uses an MILP-based assignment method to optimise QPU allocation for each circuit QCirc_m, taking into account the available QPUs, network topology, and link characteristics. The key difference from the batch-QCirc scheduling method described in the previous subsection is that the batch approach jointly optimises allocation for all circuits in a batch, whereas the single-circuit method follows a one-by-one, online strategy that locally optimises each assignment. As a result, diversity in circuit structures and characteristics across the batch is not fully exploited in this method. On the other hand, the single-QCirc approach offers lower computational complexity and enables immediate scheduling of each circuit as soon as the required resources become available.

We adopt the same MILP-based framework to determine the optimal allocation for each circuit. This assignment procedure

is referred to as ASSIGNQCIRC in Algorithm 4, and its MILP formulation is provided in Formulation 2. The same objective function and constraints from the batch formulation are used, but simplified for the single-circuit case. Note that ASSIGNQCIRC is also used within the FILLQPU procedure described in Algorithm 3.

Formulation 2. MILP-based Single-QCirc assignment problem

$$\begin{aligned} \min \quad & \left\{ \sum_{1 \leq j_1 < j_2 \leq J} z_{mj_1j_2} \left(\omega_0 \frac{w_m T_{j_1j_2}}{T_{\text{dec}}} + \omega_1 (1 - F_{j_1j_2}) \right) \right\} \\ \text{s.t.} \quad & \\ (1) \quad & \sum_{j=1}^J r_{mj} s_j N_j \geq w_m, \quad (2) \quad \sum_{j=1}^J r_{mj} \leq K_m^{\text{max}} \\ (3) \quad & r_{mj_1} + r_{mj_2} - 1 \leq z_{mj_1j_2} \leq r_{mj_1}, r_{mj_2}, \quad \forall j_1 < j_2 \end{aligned}$$

IV. EVALUATION AND RESULTS

This section evaluates the proposed scheduling algorithms through simulations. The scheduling methods (Algorithms 1 and 4) are implemented in Python. The MILP optimisation problems defined in Problem Formulations 1 and 2 are solved using the Python-MIP package with the Gurobi optimiser. The batch selection parameter β in Algorithm 1 is set to 0.85, and the maximum partition count threshold K_m^{max} is fixed at 4 for all circuits.

We first describe the evaluation setup, including the network model, benchmark circuits, workload scenarios, and scheduling-partitioning configurations. We then present and analyse the corresponding simulation results.

A. Evaluation Setup

1) *Network Model:* We consider a DQC network comprising 16 QPUs within a data centre. The QPUs are heterogeneous, with four QPUs of capacity 8 qubits, four of capacity 12 qubits, four of capacity 16 qubits, and four of capacity 20 qubits. These capacities are randomly assigned to individual QPUs using a fixed random seed to ensure consistent QPU configurations across all simulations. The entire simulation process is repeated for 10 different seed values, and the average results are presented.

The network adopts a fat tree topology of four pods with each pod consisting of 4 QPUs, as illustrated in Fig. 3(a). Since fibre lengths within a data centre are short (ranging from a few metres to hundreds of metres), propagation loss and delay are negligible and the primary source of link loss is assumed to be the optical switches. For a link traversing n_s intermediate switches, the total link transmission efficiency is approximated as $\eta_{\text{link}} \approx \eta_s^{n_s}$, where η_s is the per-switch transmission efficiency. In Sec. II, we have defined two key parameters characterising the links in the logical topology: time required to establish entanglement ($T_{j_1j_2}$) and fidelity ($F_{j_1j_2}$) of entanglement generation for remote gates. Here, we specify their values for the 4-pod fat-tree topology used in

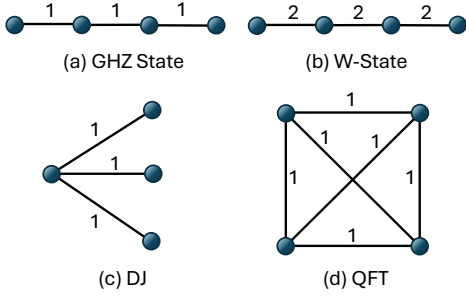


Fig. 6. Connectivity graphs for benchmark circuits ($w = 4$). Edge weights indicate the number of two-qubit gate occurrences.

our simulations. This topology features three types of links, distinguished by the number of optical switches along the path, specifically, 1, 3, or 5 switches. We denote the corresponding latency values by T_{link_1} , T_{link_3} , and T_{link_5} , respectively. Assuming an elementary link latency of T_{el} (corresponding to entanglement generation time between QPUs without loss from any optical switches), and noting that the entanglement generation rate is directly affected by link loss, the latency for each link type is calculated as:

$$T_{\text{link}_{n_s}} = \frac{T_{\text{el}}}{\eta_s^{n_s}}, \quad (8)$$

where $n_s \in \{1, 3, 5\}$. In our simulations, unless otherwise stated, all optical switches are assumed to be of the same type with an identical loss of 0.5 dB. For the fidelity parameter, we assume a slight fidelity degradation with increasing link distance and loss, as follows: $F_{\text{link}_1} = 0.96$, $F_{\text{link}_3} = 0.94$, and $F_{\text{link}_5} = 0.92$.

2) *Scheduling-Partitioning Configurations*: We evaluate the proposed algorithms under two categories of scheduling-partitioning configurations.

(a) **K-L-based scheduling-partitioning schemes**: In the first category, we consider four scheduling strategies:

- **Random Baseline (R-B)**: Circuits are randomly assigned to available QPUs subject to capacity feasibility.
- **Capacity-Aware Baseline (CA-B)**: This baseline formulates QPU allocation as a classical knapsack problem [32], accounting for QPU capacity constraints and circuit qubit demand while ignoring quantum-specific costs such as circuit connectivity, link fidelity, and decoherence. Assuming all-to-all network connectivity, it minimises the number of allocated QPUs subject to capacity and K_{max} constraints. The problem is solved to optimality using the same Gurobi solver as our proposed methods, providing a fair basis for comparison.
- **Single-QCirc**: The scheduler presented in Sec.III-B, which optimises QPU allocation for each circuit individually.
- **Batch-QCirc**: The scheduler presented in Sec.III-A, which jointly optimises QPU allocation and partition counts for a batch of circuits.

For each QCirc, once QPU allocation is determined by the scheduling policy, circuit partitioning is performed using a k -way graph partitioning approach based on iterative applications of the K-L algorithm. In what follows these three

scheduling-partitioning settings are referred to as **R-B+K-L**, **CA-B+K-L**, **Single+K-L**, and **Batch+K-L**.

(b) **Pytket-dqc-based scheduling-partitioning schemes**:

In the second category, we adopt the pytket-dqc compiler framework [24], which supports circuit placement and partitioning over heterogeneous quantum networks using advanced compilation techniques. Unlike our formulation, pytket-dqc optimises individual circuits and does not explicitly consider multi-circuit scheduling under resource contention.

We consider two pytket-dqc distributors: *PartitioningAnnealing* (PA), which employs simulated annealing on the circuit hypergraph representation, and *PartitioningHeterogeneous* (PH), which uses the KaHyPar hypergraph partitioner. In all configurations, the selected distributor determines qubit placement and circuit partitioning, and the resulting allocation is translated into a distributed circuit using pytket-dqc's standard compilation workflow.

Based on this integration, we evaluate six scheduling-partitioning configurations:

- **Pytket-PA**: Each QCirc is processed sequentially using pytket-dqc's native placement and partitioning mechanisms with the PA distributor.
- **Pytket-PH**: Same as above, but using the PH distributor.
- **Single+PA**: QPU allocation and scheduling decisions are determined by the proposed Single-QCirc scheduler, while pytket-dqc performs partitioning restricted to the subset of QPUs allocated to each circuit using PA.
- **Single+PH**: Same as above, but using PH.
- **Batch+PA**: QPU allocation and scheduling decisions are determined by the proposed Batch-QCirc scheduler, after which pytket-dqc performs partitioning within the allocated QPU subset using PA.
- **Batch+PH**: Same as above, but using PH.

In the latter four configurations, the network graph supplied to pytket-dqc is restricted to the QPUs selected by the scheduler, thereby cleanly separating high-level scheduling decisions from compiler-level partitioning. The two Pytket-only configurations serve as state-of-the-art compiler baselines, enabling direct comparison between standalone compiler-driven distribution and the proposed approaches.

3) *Benchmark QCircs and Workload Scenarios*: To establish the set of quantum circuits $\mathcal{QC}$, we use benchmark circuits from the Munich Quantum Toolkit [33]: Quantum Fourier Transform (QFT), Deutsch-Jozsa (DJ), W-state, and GHZ-state circuits. Fig. 6 shows connectivity-graph representations for $w = 4$ qubits. From this figure, it can be inferred that GHZ is sparsely connected, W-state exhibits low-to-moderate connectivity, DJ circuits exhibit an intermediate level of connectivity, and QFT circuits, modelled with a complete graph, are highly connected.

A set $\mathcal{QC}$ of size M includes all four circuit types, with each type contributing 25% of the total. For circuit QCirc_m , the number of qubits w_m is randomly selected from a type-specific range. Two workload scenarios are considered to evaluate performance under increasing qubit demand. In *Scenario 1* (Sc.1), the required qubit ranges are $R_{\text{GHZ}} = R_{\text{WState}} = [18, 26]$, $R_{\text{DJ}} = [14, 22]$, and $R_{\text{QFT}} = [10, 18]$, where smaller ranges are assigned to QFT and DJ to account for their

higher connectivity requirements. In *Scenario 2 (Sc.2)*, the qubit demand is increased by shifting each range upward by 4, yielding $R_{\text{GHZ}} = R_{\text{WState}} = [22, 30]$, $R_{\text{DJ}} = [18, 26]$, and $R_{\text{QFT}} = [14, 22]$. Simulations are conducted for $M \in \{12, 20, 28, 36\}$, and for each value of M , the experiment is repeated over 10 independently generated $\mathcal{QC}$ sets with results averaged across runs.

4) *Figures of merit*: To evaluate the performance of the proposed scheduling methods, we consider several key figures of merit. All results are averaged over repeated simulation runs.

- **Number of ebits per QCirc**: the total number of ebits across all M QCircs, divided by M , representing the quantum communication overhead.
- **Number of circuit partitions per QCirc**: the total number of circuit partitions across all M QCircs, divided by M .
- **Normalised Job Execution Time (JET) for each QCirc type**: Average execution time for a given QCirc type, computed over all its occurrences within $\mathcal{QC}$ and normalised by T_{dec} , i.e., $\text{JET}/T_{\text{dec}}$.
- **Number of ebits for each QCirc type**: Average number of ebits for a given QCirc type, computed over all its occurrences within $\mathcal{QC}$.
- **Normalised Makespan**: the total time required to complete all jobs (i.e., the execution of all M QCircs), normalised by T_{dec} , for consistency with normalised JET and to ensure hardware platform independence.
- **Normalised Throughput**: defined as the ratio of the number of QCircs (M) to Normalised Makespan. This indicates how many QCircs can be executed per unit time.

The normalised JET for a given QCirc is computed based on a layered circuit model, where the circuit is represented as a sequence of layers, and each layer comprises quantum gates that can be executed in parallel. First, for a given QCirc, remote entanglement generation is assumed to occur sequentially; parallel entanglement generation is not considered, even when multiple remote gates appear within the same circuit layer. Second, all local gates are assumed to have identical execution durations. Under these assumptions, we iterate over all circuit layers and determine whether each layer contains remote gates or only local gates. Let N_{LL} denote the number of layers containing exclusively local gates. The normalised JET is then computed as follows:

$$\frac{\text{JET}}{T_{\text{dec}}} = N_{\text{LL}} \frac{T_{\text{local}}}{T_{\text{dec}}} + \sum_{n_s \in \{1, 3, 5\}} N_{\text{link}_{n_s}}^{(e)} \frac{T_{\text{link}_{n_s}}}{T_{\text{dec}}} \quad (9)$$

where $N_{\text{link}_{n_s}}^{(e)}$ denotes the number of ebits in the QCirc under consideration that are generated over links containing n_s optical switches. In all simulations, we assume $T_{\text{local}}/T_{\text{dec}} = 5 \times 10^{-4}$ and $T_{\text{el}}/T_{\text{dec}} = 0.005$.

B. Regression-Based Estimation of ν_{mk}

This subsection presents the implementation details and performance evaluation of the linear regression model used to estimate the partitioning cost coefficient ν_{mk} . The model is

TABLE II
LINEAR REGRESSION RESULTS FOR OUR BENCHMARK CIRCUITS.

	$\hat{X}_{0,k}$	$\hat{X}_{1,k}$	$\hat{X}_{2,k}$	$\hat{X}_{3,k}$	R_{test}^2	$\text{RSME}_{\text{test}}$
$k = 2$	0.0272	0.4345	0.0163	0.0434	0.995	0.0175
$k = 3$	0.1185	0.4808	0.0534	0.0802	0.986	0.0369
$k = 4$	0.1887	0.4585	0.081	0.1235	0.973	0.055
$k = 5$	0.2842	0.3836	0.119	0.162	0.953	0.075
$k = 6$	0.368	0.3	0.152	0.0203	0.925	0.096

implemented using the `scikit-learn` library, while graph-related computations are performed with `NetworkX`.

The training dataset consists of all four circuit types with qubit counts $w_m \in [10, 30]$ (84 circuits). For each circuit, the connectivity features γ_m , $\lambda_{2,m}$, and σ_m/g_m are extracted from its graph representation. The regression target (see (4)) is obtained by computing the balanced k -way graph cut size via iterative applications of the Kernighan–Lin (K–L) algorithm. The estimated partitioning cost ν_{mk} is then computed using the linear model defined in (5).

For evaluation, a separate test set comprising all four circuit types with $w_m \in [31, 40]$ (40 circuits) is used. The learned coefficients, along with the test-set R^2 scores and root mean square error (RMSE), are reported in Table II. The results demonstrate strong predictive performance across different partition counts. Among the features, $\lambda_{2,m}$ has the strongest influence, whereas the coefficients associated with γ_m and σ_m/g_m are generally smaller. As the partition count k increases, the influence of γ_m and σ_m/g_m becomes more pronounced. Although prediction accuracy slightly decreases with larger k , the R^2 score remains above 0.9 in all cases. Since the test set contains circuits with qubit counts strictly outside the training range, the consistently high R^2 scores suggest that the model generalises well to unseen circuit sizes.

Note that generalisation to circuit types not present in the training set is not evaluated here, as the model is intended for use with the specific circuit types considered in this work. A more comprehensive study involving diverse circuit types and more sophisticated regression models is left for future work.

C. Results for K-L-based scheduling-partitioning schemes

In this subsection, we present the simulation results for the K-L-based scheduling-partitioning schemes. Fig. 7 shows the average number of ebits and circuit partitions per QCirc for the proposed and baseline methods. The results are reported for the qubit-range schemes Sc.1 and Sc.2 used in our simulations. For the Batch-QCirc scheduling method, several values of the parameter α (the fraction threshold of the total qubits available across all QPUs before executing the next batch) are considered. As shown, both the Batch-QCirc and Single-QCirc methods outperform the baseline schemes. Moreover, the Batch-QCirc approach achieves better performance than the Single-QCirc method, albeit at the cost of a more complex MILP optimisation process. Since the number of ebits directly reflects the demand for inter-QPU communication resources, these results demonstrate that both proposed methods, and particularly the Batch-QCirc assignment, effectively reduce inter-QPU communication overhead.

It can also be observed that the Random Baseline (R-B) consistently results in substantially higher ebit and partition

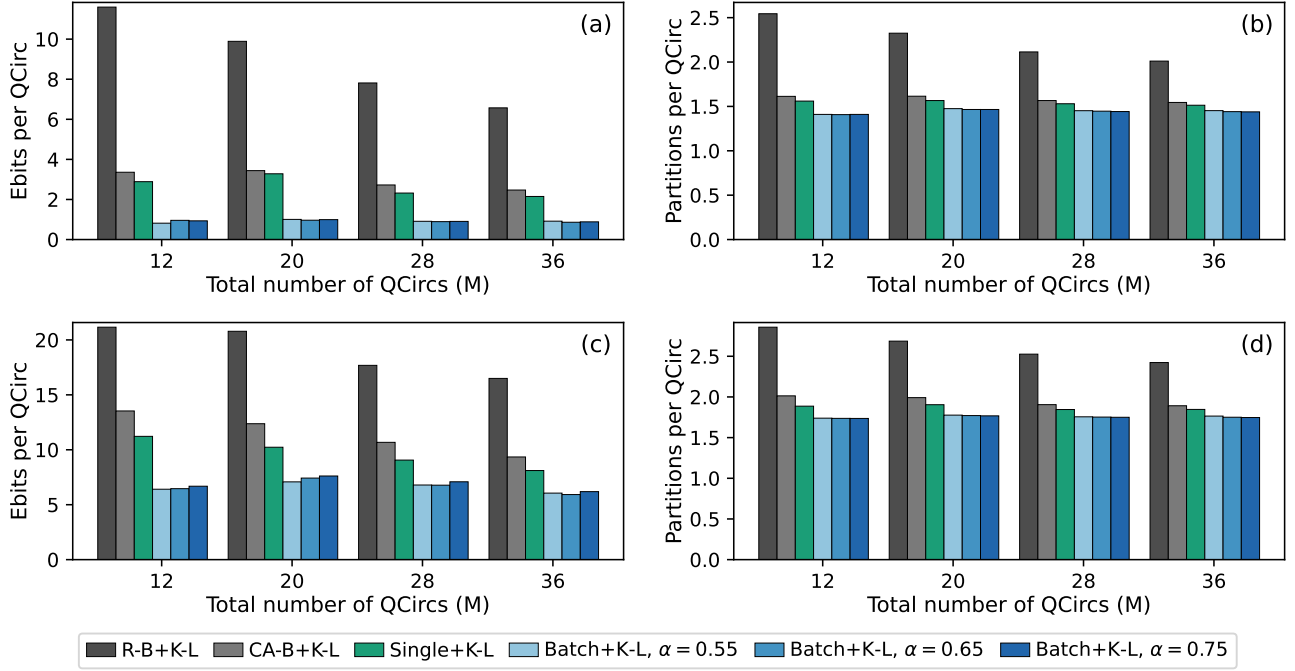


Fig. 7. Average number of ebits and circuit partitions divided by the total number of QCircs M , for the proposed and baseline scheduling-partitioning schemes. (a)-(b) Qubit-range scheme Sc.1, (c)-(d) Qubit-range scheme Sc.2.

counts than Capacity-Aware Baseline (CA-B) and the proposed methods. While it is included in Fig. 7 for completeness, it is excluded from the subsequent figures to maintain clarity and appropriate scaling.

Next, we evaluate the normalised Job Execution Time (JET), computed individually for each circuit type. As shown in Fig. 8, the proposed methods consistently outperform CA-B, achieving reduced execution times across all QCirc types. The Batch-QCirc scheduling approach further improves JET in most cases compared to the Single-QCirc method. In particular, Batch-QCirc significantly reduces the execution time for circuits with highly connected structures, such as the QFT circuits considered in our simulations, under both Sc.1 and Sc.2. In the joint-optimisation-based Batch-QCirc scheduling framework, QPUs with greater capacity tend to be prioritised for circuits with higher connectivity, thereby improving fidelity and reducing inter-QPU communication. However, this may limit optimisation opportunities for less connected circuits, such as GHZ. This trade-off is evident in both Sc.1 and Sc.2, where the Single-QCirc approach occasionally achieves lower JET for GHZ circuits. Such interdependence highlights the system-level nature of Batch-QCirc scheduling, where resource trade-offs naturally arise.

Next, we examine the normalised makespan and throughput, which are key metrics for evaluating scheduling efficiency and resource utilisation. The results are shown in Fig. 9. The proposed methods improve both metrics, with the Batch-QCirc approach generally outperforming the Single-QCirc method. These results suggest that reductions in the required number of ebits can contribute to shorter makespan and higher throughput under the evaluated settings. We also compare Batch-QCirc scheduling with varying values of α , examining all the results presented in this section. In terms of makespan and throughput,

the Batch-QCirc scheduling with $\alpha = 0.75$ consistently results in a slightly worse performance compared to other values. Since the parameter α determines when the algorithm proceeds to the next scheduling cycle, it plays a crucial role in affecting these evaluated metrics. For the results of average number of ebits and JET, similarly, $\alpha = 0.75$ does not show meaningful improvements over $\alpha = 0.55$ or $\alpha = 0.65$; In fact, the latter two often outperform $\alpha = 0.75$. These results suggest that increasing waiting time for scheduling the next batch with a higher α may lead to unnecessary resource waste without performance gains.

To investigate the impact of optical switch on the performance of the proposed DQC scheme, we increase the optical switch loss, which consequently raises the average entanglement generation time - $T_{\text{link}_1}, T_{\text{link}_3}, T_{\text{link}_5}$. Fig. 10 presents the makespan results for the CA-B, Single-QCirc and Batch-QCirc ($\alpha = 0.55$) methods under varying optical switch losses, $\eta_s = 0.5$ dB, 1 dB, and 2 dB, assuming the qubit-range scheme Sc.2. The results demonstrate that as η_s increases, the relative advantage of Batch-QCirc becomes more pronounced. For instance, for $M = 36$, Batch-QCirc reduces the makespan by 14.7% compared to Single-QCirc at $\eta_s = 0.5$ dB, with the improvement increasing to 28.3% and 43.8% at $\eta_s = 1$ dB and $\eta_s = 2$ dB, respectively. These results highlight the particular promise of the Batch-QCirc approach in topologies with heterogeneous link qualities.

D. Results for pyket-dqc-based scheduling-partitioning schemes

In this subsection, we present the simulation results for the pyket-dqc-based scheduling-partitioning schemes. In addition to Sc.1 and Sc.2, we introduce an additional scenario, Sc.3, representing a partition-intensive regime with qubit ranges

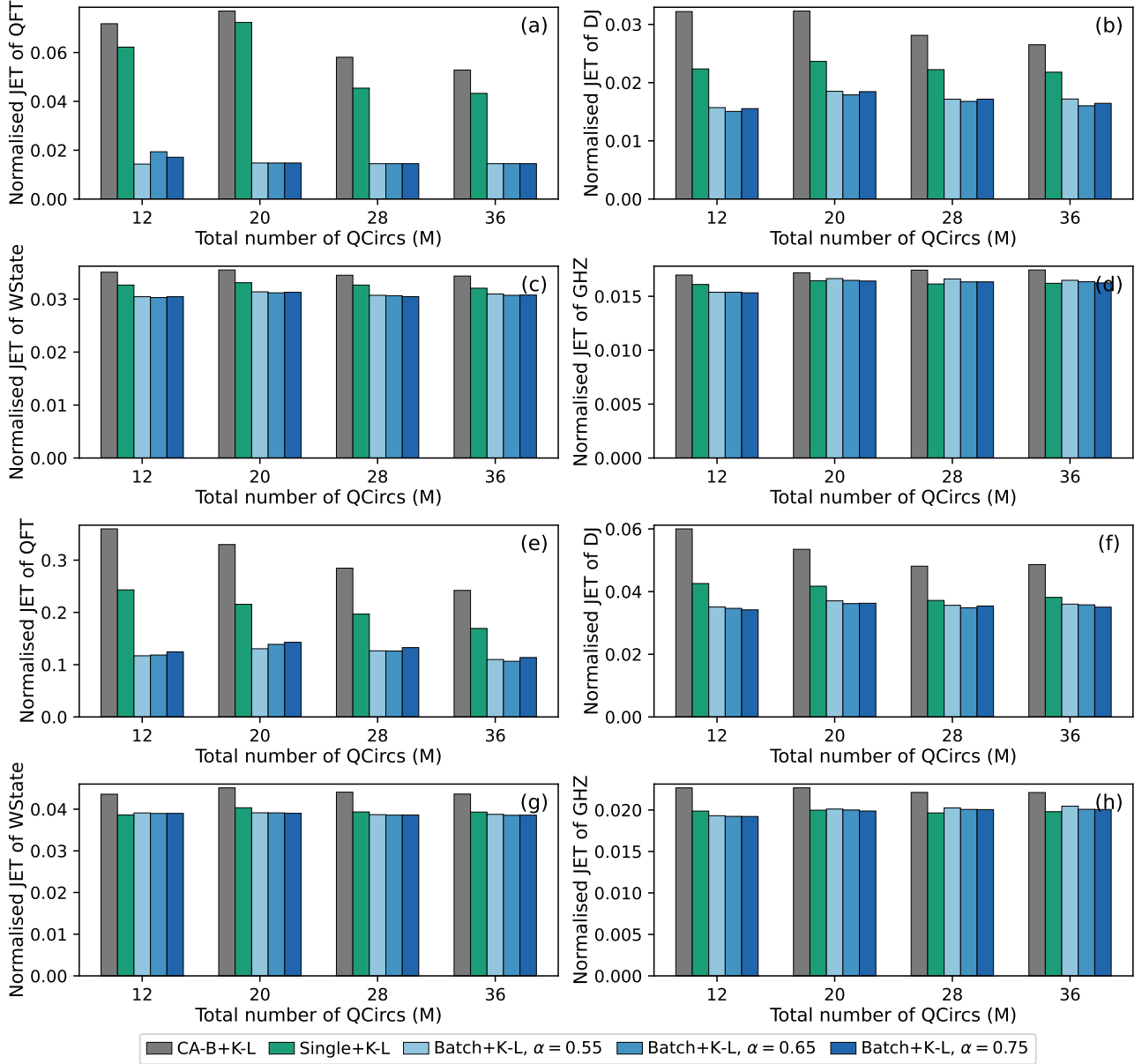


Fig. 8. Average Job Execution Time (JET) (normalised by T_{dec}) for the proposed and baseline scheduling-partitioning schemes across different circuit types. (a)–(d) Qubit-range scheme Sc.1, (e)–(h): Qubit-range scheme Sc.2.

$R_{\text{QFT}} = R_{\text{DJ}} = [22, 30]$ and $R_{\text{GHZ}} = R_{\text{WState}} = [24, 32]$. In Sc.3, all circuits exceed the maximum single-QPU capacity of 20 qubits, ensuring that distributed execution with inter-QPU communication is unavoidable. This scenario is therefore designed to stress-test the combined scheduling-partitioning pipeline under fully distributed execution conditions. In our simulations, we consider $M \in \{12, 20, 28\}$.

Figure 11 illustrates representative results, including average ebits per QCirc, average partition count per QCirc, and normalised makespan, for all six pytket-dqc-based scheduling-partitioning schemes under Sc.1, Sc.2, and Sc.3. The results demonstrate that both proposed methods consistently reduce ebit consumption and partition counts compared to the baselines (Pytket-PA and Pytket-PH). Comparing Single-QCirc and Batch-QCirc within these schemes, Batch-QCirc achieves lower ebit consumption and partition counts across

all scenarios. However, Single-QCirc yields slightly shorter makespan for $M = 20$ and $M = 28$. This highlights an important system-level trade-off: while Batch-QCirc reduces communication overhead and JET through joint optimisation across circuits, it introduces scheduling latency governed by α , whereas Single-QCirc performs scheduling immediately upon circuit arrival.

TABLE III
AVERAGE NUMBER OF EBITS PER QCIRC TYPE, ASSUMING $M = 28$ AND THE PH DISTRIBUTOR.

	PH			Single+PH			Batch+PH		
	Sc.1	Sc.2	Sc.3	Sc.1	Sc.2	Sc.3	Sc.1	Sc.2	Sc.3
QFT	1.375	6.01	17.2	1.04	4.48	13.35	0	2.23	10.53
DJ	0.96	1.93	2.82	0.47	0.88	1.15	0.26	0.67	1.01
W-State	3.39	5.15	5.4	1.68	2.25	2.39	1.52	2.1	2.67
GHZ	1.67	2.7	2.75	0.77	1.15	1.19	0.82	1.09	1.27

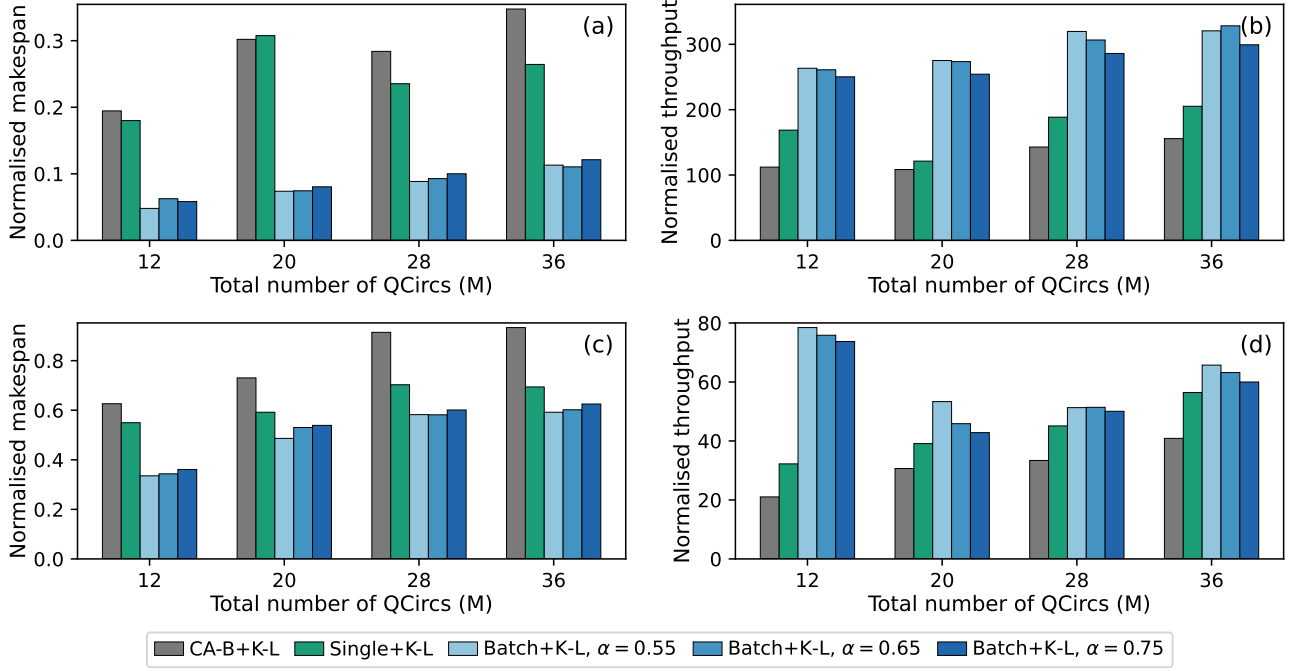


Fig. 9. Average makespan and throughput (normalised by T_{dec}) for the proposed and baseline scheduling-partitioning schemes. (a)-(b) Qubit-range scheme Sc.1, (c)-(d) Qubit-range scheme Sc.2.

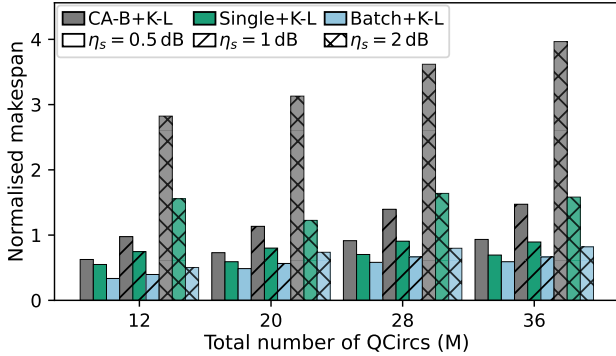


Fig. 10. Average makespan (normalised by T_{dec}) for the proposed and baseline scheduling-partitioning schemes under varying optical switch loss, assuming the qubit-range scheme Sc.2 and $\alpha = 0.55$.

To further examine the behaviour of the proposed methods, Table III reports the average number of ebits for each of the four circuit types individually in the case $M = 28$ using the PH distributor. The results show that both proposed approaches reduce ebit consumption compared to the baseline, with Batch-QCirc consistently achieving the lowest values. This indicates that, under the evaluated settings, the observed makespan trade-off primarily arises from scheduling latency rather than increased communication overhead associated with specific circuit types.

E. Computational Complexity and MILP Solver Runtime

In this subsection, we analyse the computational complexity of the proposed MILP formulations and discuss their scalability. Let J denote the number of QPUs, S the batch size, and $K = \max_m K_m^{\text{max}}$. The linearisation introduces auxiliary variables over QPU pairs, where $P = \binom{J}{2}$. Formulation 1

(Batch-QCirc) contains $S(J + K + P + KP)$ binary variables and $1 + 2S + J + 3SP + 3SKP$ linear constraints, scaling as $\Theta(SKJ^2)$. Formulation 2 (Single-QCirc) contains $J + P$ binary variables and $2 + 3P$ constraints, scaling as $\Theta(J^2)$. For the simulation setting $J = 16$ and $K = 4$, Formulation 2 has 136 binary variables and 362 constraints, while Formulation 1 has $620S$ binary variables and $(17 + 1802S)$ constraints.

The expected batch size S depends on the scheduling parameters α and β . Since Batch-QCirc is triggered when an α -fraction of total qubit capacity becomes available, the expected batch size can be approximated as:

$$S_{\text{typ}} \approx \left\lfloor \frac{\beta \alpha c_{\text{tot}}}{\mathbb{E}[w_m]} \right\rfloor. \quad (10)$$

For $c_{\text{tot}} = 224$, $\beta = 0.85$, and $\mathbb{E}[w_m] \approx 23$ (Sc.2), this yields $S_{\text{typ}} \approx 4$ for $\alpha = 0.55$.

Table IV reports the average MILP solver runtime per call for Single-QCirc and Batch-QCirc ($\alpha = 0.55$) under Sc.1, Sc.2, and Sc.3. All experiments were conducted on a Ubuntu server equipped with an Intel Core Processor at 2.40 GHz, 32 GB RAM, running Gurobi v11.0.0. As expected, Batch-QCirc exhibits significantly higher runtime than Single-QCirc due to its larger joint optimisation problem. In contrast, the runtime of Single-QCirc remains relatively stable across scenarios, consistent with its fixed per-circuit problem size. For Batch-QCirc, the runtime under Sc.1 is higher than under Sc.2, since the lower average qubit demand in Sc.1 leads to larger batch sizes and therefore larger MILP instances. Interestingly, Sc.3 yields the highest Batch-QCirc runtime despite having smaller batch sizes. This is because, in Sc.3, all circuits exceed single-QPU capacity, forcing partitioning for every circuit and thereby expanding the feasible decision space explored by the MILP solver. These results indicate that the runtime of

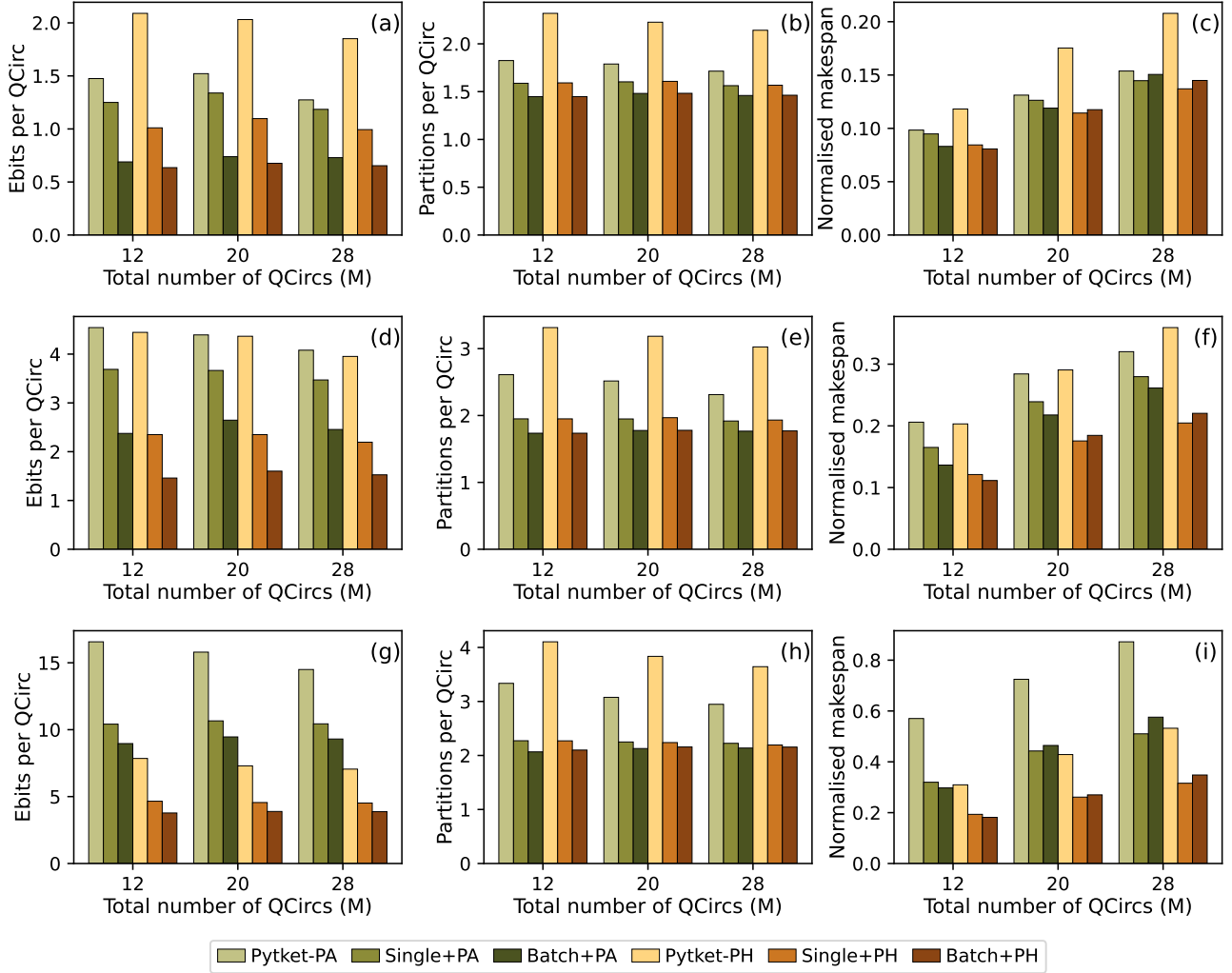


Fig. 11. Average number of ebits and circuit partitions divided by the total number of QCircs M , and Average makespan (normalised by T_{dec}) for the proposed and baseline Pytket-dqc-based scheduling-partitioning schemes. (a)-(c) Qubit-range scheme Sc.1, (d)-(f) Qubit-range scheme Sc.2, (g)-(i) Qubit-range scheme Sc.3.

TABLE IV
AVERAGE MILP SOLVER RUNTIME (S).

	Sc.1	Sc.2	Sc.3
Single-QCirc	0.0062	0.0061	0.0065
Batch-QCirc ($\alpha = 0.55$)	3.41	2.93	4.14

Batch-QCirc is influenced not only by batch size but also by circuit demand characteristics and partitioning intensity. While the runtimes observed here are acceptable for the considered network scale, extending the approach to substantially larger systems may benefit from heuristic or decomposition-based strategies, which we identify as a direction for future work.

V. CONCLUSION

In this work, we address the challenge of efficient resource management and QCirc scheduling for DQC. We propose two scheduling approaches: a dynamic Batch-QCirc scheduling method, which operates through sequential scheduling cycles with batch optimisation, and a Single-QCirc scheduling method that optimises QCirc allocation individually. To achieve optimal assignment of QCircs to QPUs, we develop

MILP formulations that minimise errors arising from inter-QPU communication. These formulations explicitly account for network topology, link loss, QPU capacities, and QCirc structure, enabling communication- and resource-aware allocation decisions. More specifically, the Batch-QCirc approach enables system-level optimisation across multiple concurrent circuits, thereby reducing communication overhead and improving execution fidelity, especially in heterogeneous network settings. At the same time, scalability considerations, together with the observed trade-off between inter-QPU communication overhead and makespan, suggest that batch-based optimisation is especially advantageous in small- to medium-scale, communication-resource-constrained regimes. On the other hand, the Single-QCirc formulation offers a lightweight and responsive scheduling strategy more suitable for online scheduling.

This work represents an important step toward quantum- and network-aware resource management in cloud-based quantum data centres. To extend this further, consideration of dynamic quantum link models, heterogeneous hardware characteristics, and larger-scale deployments presents as important direc-

tions for future work. In addition, incorporating priority- and deadline-aware scheduling policies, as well as exploring machine learning-based approaches for adaptive and scalable optimisation, could further enhance intelligent resource management in quantum cloud infrastructures.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to Reza Nejabati for his support and insightful technical discussions.

REFERENCES

- [1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.
- [2] M. Caleffi, M. Amoretti, D. Ferrari, J. Illiano, A. Manzalini, and A. S. Cacciapuoti, "Distributed quantum computing: a survey," *Computer Networks*, vol. 254, p. 110672, 2024.
- [3] A. Yimsiriwattana and S. J. Lomonaco Jr, "Distributed quantum computing: A distributed shor algorithm," in *Quantum Information and Computation II*, vol. 5436. SPIE, 2004, pp. 360–372.
- [4] D. Barral, F. J. Cardama, G. Diaz-Camacho, D. Faílde, I. F. Llovo, M. Mussa-Juane, J. Vázquez-Pérez, J. Villasuso, C. Piñeiro, N. Costas *et al.*, "Review of distributed quantum computing: from single qpu to high performance quantum computing," *Computer Science Review*, vol. 57, p. 100747, 2025.
- [5] K. Campbell, A. Lawey, and M. Razavi, "Quantum data centres: a simulation-based comparative noise analysis," *Quantum Science and Technology*, vol. 10, no. 1, p. 015052, 2024.
- [6] A. Hussain, M. Aleem, A. Khan, M. A. Iqbal, and M. A. Islam, "RALBA: a computation-aware load balancing scheduler for cloud computing," *Cluster Computing*, vol. 21, pp. 1667–1680, 2018.
- [7] S. Singhal and A. Sharma, "Resource scheduling algorithms in cloud computing: A big picture," in *2021 5th International Conference on Information Systems and Computer Networks (ISCON)*. IEEE, 2021, pp. 1–6.
- [8] S. K. Mishra, B. Sahoo, and P. P. Parida, "Load balancing in cloud computing: a big picture," *Journal of King Saud University-Computer and Information Sciences*, vol. 32, no. 2, pp. 149–158, 2020.
- [9] O. Elzeki, M. Rashad, and M. A. Elsoud, "Overview of scheduling tasks in distributed computing systems," *International Journal of Soft Computing and Engineering*, vol. 2, no. 3, pp. 470–475, 2012.
- [10] R. D. Oliveira, S. Bahrani, E. Arabul, R. Wang, R. Nejabati, and D. Simeonidou, "FPGA-based deterministic and low-latency control for distributed quantum computing," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2023, pp. 1–6.
- [11] F. Vista, S. DiAdamo, E. Kaur, H. Shapourian, and R. Nejabati, "Entanglement request scheduling in quantum datacenter networks," *IEEE Network*, 2025.
- [12] H. Zhang, Y. Xu, H. Hu, K. Yin, H. Shapourian, J. Zhao, R. R. Kompella, R. Nejabati, and Y. Ding, "Switchqnet: Optimizing distributed quantum computing for quantum data centers with switch networks," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1449–1463.
- [13] S. Pouryousef, R. Nejabati, D. Towsley, R. Kompella, and E. Kaur, "Network-aware scheduling for remote gate execution in quantum data centers," *arXiv preprint arXiv:2504.20176*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.20176>
- [14] C. Cicconetti, M. Conti, and A. Passarella, "Resource allocation in quantum networks for distributed quantum computing," in *2022 IEEE International conference on smart computing (SMARTCOMP)*. IEEE, 2022, pp. 124–132.
- [15] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.
- [16] D. Cuomo, M. Caleffi, K. Krsulich, F. Tramonto, G. Agliardi, E. Prati, and A. S. Cacciapuoti, "Optimized compiler for distributed quantum computing," *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–29, 2023.
- [17] D. Ferrari, S. Carretta, and M. Amoretti, "A modular quantum compilation framework for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, 2023.
- [18] O. Daei *et al.*, "Optimized quantum circuit partitioning," *International Journal of Theoretical Physics*, vol. 59, no. 12, pp. 3804–3820, 2020.
- [19] W. Cambiucci *et al.*, "Hypergraphic partitioning of quantum circuits for distributed quantum computing," *arXiv preprint arXiv:2301.05759*, 2023.
- [20] P. Andres-Martinez *et al.*, "Automated distribution of quantum circuits via hypergraph partitioning," *Physical Review A*, vol. 100, no. 3, p. 032308, 2019.
- [21] Z. Davarzani, M. Zomorodi-Moghadam, M. Houshmand, and M. Nouri-Baygi, "A dynamic programming approach for distributing quantum circuits by bipartite graphs," *Quantum Information Processing*, vol. 19, pp. 1–18, 2020.
- [22] E. Kaur, H. Shapourian, J. Zhao, M. Kilzer, R. Kompella, and R. Nejabati, "Optimized quantum circuit partitioning across multiple quantum processors," in *Quantum Computing, Communication, and Simulation V*, vol. 13391. SPIE, 2025, pp. 152–156.
- [23] D. Dadkhah, M. Zomorodi, and S. E. Hosseini, "A new approach for optimization of distributed quantum circuits," *International Journal of Theoretical Physics*, vol. 60, pp. 3271–3285, 2021.
- [24] P. Andres-Martinez, T. Forrer, D. Mills, J.-Y. Wu, L. Henaut, K. Yamamoto, M. Murao, and R. Duncan, "Distributing circuits over heterogeneous, modular quantum computing network architectures," *Quantum Science and Technology*, vol. 9, no. 4, p. 045021, 2024.
- [25] R. Parekh *et al.*, "Quantum algorithms and simulation for parallel and distributed quantum computing," in *2021 IEEE/ACM Second International Workshop on Quantum Computing Software (QCS)*. IEEE, 2021, pp. 9–19.
- [26] H. Shapourian *et al.*, "Quantum data center infrastructures: A scalable architectural design perspective," *arXiv preprint arXiv:2501.05598*, 2025.
- [27] J. Ang, G. Carini, Y. Chen, I. Chuang, M. Demarco, S. Economou, A. Eickbusch, A. Faraon, K.-M. Fu, S. Girvin *et al.*, "Arquin: architectures for multinode superconducting quantum computers," *ACM Transactions on Quantum Computing*, vol. 5, no. 3, pp. 1–59, 2024.
- [28] H. K. Beukers, M. Pasini, H. Choi, D. Englund, R. Hanson, and J. Borregaard, "Remote-entanglement protocols for stationary qubits with photonic interfaces," *PRX Quantum*, vol. 5, no. 1, p. 010202, 2024.
- [29] M. Maronese, L. Moro, L. Rocutto, and E. Prati, "Quantum compiling," in *Quantum Computing Environments*. Springer, 2022, pp. 39–74.
- [30] Y. Shi, P. Gokhale, P. Murali, J. M. Baker, C. Duckering, Y. Ding, N. C. Brown, C. Chamberland, A. Javadi-Abhari, A. W. Cross *et al.*, "Resource-efficient quantum computing by breaking abstractions," *Proceedings of the IEEE*, vol. 108, no. 8, pp. 1353–1370, 2020.
- [31] M. Fiedler, "Algebraic connectivity of graphs," *Czechoslovak Mathematical Journal*, vol. 23, no. 2, pp. 298–305, 1973.
- [32] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Springer, 2004.
- [33] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, 2023, MQT Bench is available at <https://www.cda.cit.tum.de/mqtbench/>.